

TRAQ: Estimating the Quantum Cost of Classical Programs*

ANURUDH PEDURI, JAM KABEER ALI KHAN, GILLES BARTHE, and MICHAEL WALTER

Predicting practical speedups offered by future quantum computers has become a major focus of the quantum computing community. Typically, these predictions are supported by lengthy manual analyses and numerical simulations and are carried out for one specific application at a time. In this paper, we present TRAQ, a principled approach towards estimating the quantum speedup of classical programs fully automatically and with provable guarantees. It consists of a classical language that includes high-level primitives amenable to quantum speedups, a cost analysis, and a compilation to low-level quantum programs. Our cost analysis upper bounds the complexity of the resulting quantum program in a fine-grained way: it captures non-asymptotic information and is sensitive to the input of the program (rather than providing worst-case costs).

1 Introduction

Quantum computing gives us access to a plethora of quantum algorithms that can offer asymptotic speedups relative to their classical (i.e. non-quantum) counterparts. Examples include Grover’s algorithm [9, 22, 25], which offers a *quadratic* speedup for unstructured search, quantum max/min-finding [19], quantum counting [12], Shor’s algorithm for factorization and discrete logarithms [34, 35], quantum algorithms for linear systems [23] and convex optimization problems [10, 40]. The Quantum Algorithm Zoo [26] is a comprehensive collection of quantum algorithms, discussing their complexity and potential speedups.

We would ideally like to take advantage of such speedups in programs. One common approach to utilizing quantum subroutines is by *quantization* [1, 39]. This refers to implementing a chosen classical subroutine using a quantum algorithm. This technique can be used in a larger context, by replacing many subroutines of a large classical program, each with their respective quantum implementation. For example, if a program that uses search, we can implement this search call using Grover’s algorithm. This approach has been widely used to develop quantum algorithms for cryptanalysis [3, 7, 8, 18, 30, 33], and NP-hard optimization problems [6, 17, 20, 27, 29, 41, 42].

A natural question to ask is whether such a quantization offers an actual quantum speedup in practice, and for what problem sizes and instances. Unlike classical programs, we often cannot yet run the new quantum program due to a lack of adequate quantum hardware. Therefore we must be able to *estimate* the cost of the new quantum program, to understand which subroutines to quantize to obtain a speedup [24, 32, 37]. In spite of lack of hardware to run the quantum programs, such estimates have many practical applications. For instance, they are used by cryptanalysts to estimate the cost benefits of making their attacks quantum — the cost estimates are used to determine the size of keys. They can also provide insights into which subroutines to quantize in a larger program to obtain better speedups. To summarize, the question we wish to address in our work is:

Can we estimate input-dependent quantum cost of quantized classical programs with provable guarantees, and without running or simulating quantum programs?

This approach has been used in prior works to study quantum speedups. An important area of study are search algorithms for cryptanalysis: pre-image attacks on SHA-2 and SHA-3 [7], lattice problems [3, 30], security of AES [8, 18], and generic nested search algorithms [33]. Other applications include SAT [13, 14, 17, 20], community detection [16], knapsack [41, 42], linear solvers [27], and simplex [6, 29]. So far, these such analyses had to be done manually by hand, which can be tedious and error-prone, and they required deep quantum expertise to arrive at the desired quantum cost estimates.

*Initial preprint: <https://arxiv.org/abs/2509.01508>. Minor notation improvements in this abstract compared to preprint.

This work. We present TRAQ, a principled framework for automating the cost analysis of quantized programs. We developed a Haskell package `traq`, which consists of an EDSL to describe classical programs with high-level quantizable primitives. We also develop a meta-theory formalizing the core language, compiler, and cost analyses. We provide formal guarantees by proving that our compiler preserves semantics up to a factor of the total error, and that our cost analyses upper-bound the actual cost of the quantum programs.

2 Background

Now, we concretely explain the precise input model we use to load data into our programs, and the cost model used by our cost analysis to bound the complexity of these programs.

Query Model. Consider a function $f : X \rightarrow Y$ for some finite sets X, Y , which encodes some problem input. A *query* to f is simply calling the it on some inputs $x \in X$, to obtain the value $f(x) \in Y$. Quantum algorithms such as Grover [22] and Simon [36] exact an advantage when they can query such a function in *superposition*. That is, they query a *unitary* U_f which has the following action:

$$U_f |x\rangle |0\rangle = |x\rangle |f(x)\rangle \quad (1)$$

for every $x \in X$. Here $|x\rangle$ is a quantum basis state vector corresponding to x . Such a model for quantumly querying a function is commonly used in many applications, e.g. in cryptography [2]. Such a unitary U_f can either be described by some quantum circuit, or loaded from data using a data-structure like a QRAM/QROM [21].

Program Cost. Our framework estimates the number of calls or queries made to each externally provided function as described above, where each such function represents some input data to the program. This approach, known as query complexity [15], yields a well-established proxy for time complexity, and is agnostic of the details of the platform (hardware, gateset etc.).

3 Overview of TRAQ

We give an overview of our framework by describing challenges in quantizing classical programs and then, explain our compiler-aware cost analysis using illustrative example.

3.1 Quantizing Classical Programs

As motivated before, in this work we study *quantized* classical programs. To express quantized classical program, we give a classical source language CPL, in which we enable quantization by providing high-level primitives, where each primitive can be implemented using either a classical algorithm or a quantum algorithm. That is, if a CPL program uses a high-level primitive, then it can be quantized.

Compilation. To quantize the classical program, we provide a compiler $Q[\cdot]$ from CPL to a classical-quantum programming language, called QPL, as our target, including appropriately compiling each high-level primitive to its underlying quantum algorithm.

Primitives. We model each primitive as a second-order function, which takes a number of functions, and arguments to solve a certain computational problem. For example, the *search* primitive, which takes a boolean predicate, and finds an input satisfying it or outputs false if no such input exists. We can utilize such a primitive to solve the SAT problem:

$$b \leftarrow \text{any}[f]$$

where $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is a SAT formula over n variables. It returns $b = 0$ if this instance is satisfiable, otherwise returns $b = 0$. Otherwise it returns $b = 0$ and an arbitrary x . This can be

implemented classically using repeated sampling, and quantumly using Grover’s search [9, 22]. Concretely, we can quantize it by compiling it to the quantum implementation.

Compilation Challenge: Accuracy. Quantum algorithms for primitives, such as Grover’s search [22] are *probabilistic* in nature, i.e. they have some failure probability. These errors add up in a program that calls multiple primitives. Therefore we must choose a failure probability for each primitive call in the program, satisfying an overall error budget, and the compilation will depend on these choices.

Compilation Challenge: Superposition Queries. Quantum algorithms such as Grover [22] and Simons [36] access their predicate in *superposition*, that is, as a unitary. Recall the search primitive, `any[f]`, which has an underlying quantum implementation of Grover’s algorithm, and hence, its predicate f is required to be unitary. To support this, we must compile these primitives such that their predicates are compiled to unitary quantum programs and hence, correctly handle *uncomputation* to ensure that the overall program is correct.

Analysis Challenge: Input-dependence. Our overall goal is to estimate the cost of these compiled programs. For many problems such as NP-hard optimization problems, the average-case complexity is often better than the worst-case complexity. For example, Grover’s search algorithm over an input space of size N with K solutions has an expected complexity of $O(\sqrt{N/K})$ but a worst-case complexity of $O(\sqrt{N})$ calls to the predicate. Therefore we are interested in estimating the expected cost of the given program on some given input. One way to obtain the expected cost is to run this on hardware, but this is infeasible on current hardware. Similarly, performing a quantum simulation of the program can be very expensive, especially when it requires many qubits. It is hence useful to find alternate ways to more efficiently compute or bound such costs.

3.2 TRAQ by Example

TRAQ overcomes the above challenges by performing a *compiler-aware source-level cost analysis*. As we study only quantized programs, we know the ideal *classical* behaviour of each primitive, as well as formulas for their expected cost based on some input-dependent parameters. We can utilize this by evaluating the source program appropriately to obtain information to compute the cost estimates of the compiled program. We also naturally account for the failure probability of each primitive in our source-level analysis.

Let us illustrate our framework with the following example boolean formula evaluation problem, inspired by a more general problem known as AND-OR trees. Despite its simplicity, this problem has received significant attention in the quantum computing literature [4, 5, 25], and reveals multiple challenges in building a quantum cost analysis framework.

Matrix Search Problem. *Given a $N \times M$ boolean matrix A , does it have a row containing all 1s? That is, compute $\text{OR}(\text{AND}(A_{0,0}, \dots, A_{0,M-1}), \dots, \text{AND}(A_{N-1,0}, \dots, A_{N-1,M-1}))$.*

We implement an algorithm in Figure 1 that solves this problem by *searching* over all rows of the matrix, and for each row, *searching* through its elements to check whether it contains a 0.

The program is parametrized by an abstract function `Matrix` which models the input matrix A . The function `IsRowAllOnes` uses a primitive `all`, which accepts a partially applied function, here of type `Fin⟨M⟩ → Bool`. Similarly, `HasAllOnesRow` uses `any` to check if there is a row of all 1s.

```
Matrix :: (Fin<N>, Fin<M>) --> Bool

IsRowAllOnes :: Fin<N> --> Bool
IsRowAllOnes i = do
  return all[Matrix(i, _)];

HasAllOnesRow :: () --> Bool
HasAllOnesRow () = do
  return any[IsRowAllOnes(_)];
```

Fig. 1. Matrix Search Program in CPL

Input-dependent Cost Analysis. We give a compiler-aware cost analysis $\widehat{\text{ExpCost}}^Q$, which maps source programs to cost expressions. As each primitive can only be realized up to some accuracy, we first annotate every primitive call in the program with a symbolic failure probability. We then use the known query cost formulas for these primitives to compositionally compute the total cost. For example, the primitive **any** has the following cost equation:

$$\widehat{\text{ExpCost}}^Q[\text{any}_\varepsilon[f(x, _)]](\sigma) = E_{\text{QSearch}}(N, K_{\hat{f}}, \varepsilon) \cdot 2 \cdot \widehat{\text{Cost}}^{\mathcal{U}}[f]$$

where $\hat{f}(v) = \llbracket f \rrbracket(\sigma(x), v)$ is the function evaluating the predicate, and $\llbracket \cdot \rrbracket$ is a deterministic source semantics. The parameter $K_{\hat{f}} = |\{v \in \{0 \dots N-1\} \mid \hat{f}(v) = 1\}|$ is the number of solutions, which we can compute using only the source semantics. The function $\widehat{\text{Cost}}^{\mathcal{U}}[f]$ is the cost of implementing f unitarily, and the factor of 2 is to account for uncomputation.

The primitive is implemented using the quantum search algorithm [11], which for a search space of N elements with K solutions, to succeeds with probability $\geq 1 - \varepsilon$, makes an expected $E_{\text{QSearch}}(N, K, \varepsilon)$ queries to a *unitary* implementing f .

Optimizing Errors. In general, we have an overall error budget $\varepsilon_{\text{total}}$, and we want to pick individual errors to satisfy this. TRAQ takes an annotated program and uses a source-level error analysis (similar to the cost analysis above) to compute a symbolic expression for the overall error. In our example, it produces

$$\text{err}^Q[\text{HasAllOnesRow}] = \varepsilon_2 + W_{\text{QSearch}}(N, \varepsilon_2) \cdot 2\sqrt{\varepsilon_1}$$

It then picks individual errors $\varepsilon_1, \varepsilon_2 \in [0, 1]$ such that the above term is bounded by $\varepsilon_{\text{total}}$. We can also further optimize these to minimize the cost expression above.

Superposition and Nested Calls. In this example, the outer **any** is implemented using quantum search by [9], which requires unitary access to its predicate. Therefore we compile `IsRowAllOnes` unitarily, using a second compiler $\mathcal{U}$. This further leads to a new challenge: the inner primitive **all** must be compiled fully unitarily. So for each primitive, we also provide a fully unitary quantum algorithm to realize it. For example, for search algorithms, we use the algorithm due to Zalka [43].

Formal Guarantees. We prove formal guarantees on the compilation and cost analysis. This ensure that cost of a quantized classical program can be soundly analyzed at the source level, without requiring compilation or execution of the target:

THEOREM 1 (QUANTUM COMPILATION PRESERVES EXPECTED COST). *For every well-formed CPL statement s and well-formed input σ ,*

$$\text{ExpCost}[Q[s]](\sigma) \leq \widehat{\text{ExpCost}}^Q[s](\sigma) + \text{err}^Q[s] \cdot \widehat{\text{Havoc}}^Q[s],$$

for quantum compiler $Q[\cdot]$, total error probability $\text{err}^Q[\cdot]$ and worst-case quantum cost $\widehat{\text{Havoc}}^Q[\cdot]$.

Intuitively, this states that the expected cost is bounded by our analysis, except for an $\varepsilon = \text{err}^Q[s]$ probability of failure, where it incurs the worst-case cost.

THEOREM 2 (QUANTUM COMPILATION PRESERVES SEMANTICS). *For every well-formed CPL statement s and well-formed input σ ,*

$$\text{TV}(\llbracket Q[s] \rrbracket(\sigma), \mathbb{1}_{\llbracket s \rrbracket(\sigma)}) \leq \text{err}^Q[s]$$

where $\llbracket \cdot \rrbracket$ is the probabilistic target semantics, and TV is the total variation distance.

Intuitively, this states that the output distribution of the compiled program is ε -close to the deterministic output of the source program (where $\mathbb{1}$ denotes the delta-distribution).

To establish both these results, we formalize the quantum and unitary compilation of source programs, and also use the correctness of the unitary compiler.

3.3 Implementation

We implemented TRAQ as a Haskell package in ~ 9000 LOCs, it provides the source-language CPL as a deeply embedded DSL, including static typing, semantics, cost analysis, and compilation to the target-language QPL. We implement the probabilistic semantics using a probability monad [31] which allows efficient sampling and computing expectations.

One of our design principles is extensibility — to support adding new primitives with ease. We enable this by implementing a growing polymorphic AST inspired by prior Haskell work on extensible ASTs [28, 38]. To give an idea, the expression AST is

```
1 data Expr P = ... | PrimCall P
2 data Program P = ...
```

where P is the current extension in use. An extension could be a single primitive or a collection of primitives, or some annotated (collection of) primitive. We use *typeclasses* to allow adding functionality to each new primitive, and generics to automatically derive functionality for collections of primitives. E.g., `Evaluate` for semantics, `ExpCost` for cost etc.

```
1 class ExpCost (AnnErr P) where
2   expCost :: (MonadCost P m) => P ->  $\Sigma$  -> m Cost
```

where `MonadCost` is a mtl-style monad class supporting computing cost. As a result, TRAQ provides full extensibility to add new primitives without changing the core framework.

We then use a simple wrapper to annotate our primitives with a failure probability, and provide their cost:

```
data AnnErr P = AnnErr Double P -- annotate primitives with failure probability.
annotateProgram :: Program P -> Program (AnnErr P)
instance (Evaluate P) => ExpCost (AnnErr P) where
  expCost (AnnErr eps p)  $\sigma$  = ...
```

To optimally pick the failure probabilities, each primitive is annotated with epsilons ε_i . Then, error and worst-case analysis is used to obtain symbolic expressions for each. Then, TRAQ solves for optimal epsilons satisfying an overall error budget, subject to minimizing the cost expression. Then, computed epsilons are substituted and used in compilation and computing expected costs of the program.

4 Discussion

In summary, we provided an overview of TRAQ, a framework for estimating the quantum cost of classical programs, with formal guarantees and practical implementation as a Haskell package.

Our Contributions. The framework is being actively developed to incorporate more primitives and language features. An initial version of this work is publicly available¹, which provides a formal development and implementation of TRAQ:

- (1) Design of a classical language, CPL, with *high-level primitives* that can be quantized, and its compilation to a classical-quantum language QPL.
- (2) Compiler- and error-aware cost to bound input-dependent expected quantum cost of compiled programs with formal guarantees on the correctness of compilation and cost analysis.
- (3) High-level search primitive with variants compiling to Grover’s search [25], brute-force search, and rejection sampling.
- (4) A Haskell package, with an EDSL to write source programs, and an extensible library to add new primitives and its evaluation.

¹Initial preprint: <https://arxiv.org/abs/2509.01508>. Minor notation improvements in this abstract compared to preprint.

References

- [1] Esma Aïmeur, Gilles Brassard, and Sébastien Gambs. 2007. Quantum clustering algorithms. In *Proceedings of the 24th International Conference on Machine Learning (Corvalis, Oregon, USA) (ICML '07)*. Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/1273496.1273497
- [2] Gorjan Alagic, David Cooper, Quynh Dang, Thinh Dang, John M. Kelsey, Jacob Lichtinger, Yi-Kai Liu, Carl A. Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, Daniel Smith-Tone, and Daniel Apon. 2022. Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process. doi:10.6028/NIST.IR.8413
- [3] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. 2020. Estimating Quantum Speedups for Lattice Sieves. In *Advances in Cryptology – ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II* (Daejeon, Korea (Republic of)). Springer-Verlag, Berlin, Heidelberg, 583–613. doi:10.1007/978-3-030-64834-3_20
- [4] Andris Ambainis. 2006. Quantum search with variable times. arXiv:quant-ph/0609168 [quant-ph]
- [5] A. Ambainis, A. M. Childs, B. W. Reichardt, R. Špalek, and S. Zhang. 2010. Any AND-OR Formula of Size N Can Be Evaluated in Time $N^{\frac{1}{2}+o(1)}$ on a Quantum Computer. *SIAM J. Comput.* 39, 6 (2010), 2513–2530. doi:10.1137/080712167 arXiv:https://doi.org/10.1137/080712167
- [6] Sabrina Ammann, Maximilian Hess, Debora Ramacciotti, Sándor P. Fekete, Paulina L. A. Goedicke, David Gross, Andreea Lefterovici, Tobias J. Osborne, Michael Perk, Antonio Rotundo, S. E. Skelton, Sebastian Stiller, and Timo de Wolff. 2023. Realistic Runtime Analysis for Quantum Simplex Computation. arXiv:2311.09995 [quant-ph]
- [7] Matthew Amy, Olivia Di Matteo, Vlad Gheorghiu, Michele Mosca, Alex Parent, and John Schanck. 2017. Estimating the Cost of Generic Quantum Pre-image Attacks on SHA-2 and SHA-3. In *Selected Areas in Cryptography – SAC 2016*, Roberto Avanzi and Howard Heys (Eds.). Springer International Publishing, Cham, 317–337.
- [8] Xavier Bonnetain, María Naya-Plasencia, and André Schrottenloher. 2019. Quantum Security Analysis of AES. *IACR Transactions on Symmetric Cryptology* 2019, 2 (Jun. 2019), 55–93. doi:10.13154/tosc.v2019.i2.55-93
- [9] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. Tight Bounds on Quantum Searching. *Fortschritte der Physik* 46, 4-5 (1998), 493–505. doi:10.1002/(SICI)1521-3978(199806)46:4/5<493::AID-PROP493>3.0.CO;2-P
- [10] Fernando GSL Brandao and Krysta M Svore. 2017. Quantum speed-ups for solving semidefinite programs. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 415–426.
- [11] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. 2002. Quantum amplitude amplification and estimation. *Contemp. Math.* 305 (2002), 53–74.
- [12] Gilles Brassard, Peter Høyer, and Alain Tapp. 1998. *Quantum counting*. Springer Berlin Heidelberg, 820–831. doi:10.1007/bfb0055105
- [13] Martijn Brehm. 2023. Quantifying quantum walk speed-ups. <https://eprints.illc.uva.nl/id/eprint/2267> Deposited: 05 Sep 2023; Last modified: 26 Sep 2023.
- [14] Martijn Brehm and Jordi Weggemans. 2024. Assessing fault-tolerant quantum advantage for k -SAT with structure. arXiv:2412.13274 [quant-ph] <https://arxiv.org/abs/2412.13274>
- [15] Harry Buhrman and Ronald de Wolf. 2002. Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science* 288, 1 (2002), 21–43. doi:10.1016/S0304-3975(01)00144-X Complexity and Logic.
- [16] Chris Cade, Marten Folkertsma, Ido Niesen, and Jordi Weggemans. 2022. Quantum Algorithms for Community Detection and their Empirical Run-times. arXiv:2203.06208 [quant-ph]
- [17] Chris Cade, Marten Folkertsma, Ido Niesen, and Jordi Weggemans. 2023. Quantifying Grover speed-ups beyond asymptotic analysis. *Quantum* 7 (Oct. 2023), 1133. doi:10.22331/q-2023-10-10-1133
- [18] Nicolas David, María Naya-Plasencia, and André Schrottenloher. 2024. Quantum impossible differential attacks: applications to AES and SKINNY. *Designs, Codes and Cryptography* 92, 3 (01 Mar 2024), 723–751. doi:10.1007/s10623-023-01280-y
- [19] Christoph Durr and Peter Hoyer. 1999. A Quantum Algorithm for Finding the Minimum. arXiv:quant-ph/9607014 [quant-ph] <https://arxiv.org/abs/quant-ph/9607014>
- [20] Vahideh Eshaghian, Sören Wilkening, Johan Åberg, and David Gross. 2024. Runtime-coherence trade-offs for hybrid SAT-solvers. arXiv:2404.15235 [quant-ph] <https://arxiv.org/abs/2404.15235>
- [21] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. 2008. Quantum Random Access Memory. *Phys. Rev. Lett.* 100 (Apr 2008), 160501. Issue 16. doi:10.1103/PhysRevLett.100.160501
- [22] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (Philadelphia, Pennsylvania, USA) (STOC '96)*. Association for Computing Machinery, New York, NY, USA, 212–219. doi:10.1145/237814.237866
- [23] Aram W. Harrow, Avinandan Hassidim, and Seth Lloyd. 2009. Quantum Algorithm for Linear Systems of Equations. *Phys. Rev. Lett.* 103 (Oct 2009), 150502. Issue 15. doi:10.1103/PhysRevLett.103.150502
- [24] Torsten Hoefler, Thomas Häner, and Matthias Troyer. 2023. Disentangling Hype from Practicality: On Realistically Achieving Quantum Advantage. *Commun. ACM* 66, 5 (April 2023), 82–87. doi:10.1145/3571725

- [25] Peter Høyer, Michele Mosca, and Ronald de Wolf. 2003. *Quantum Search on Bounded-Error Inputs*. Springer Berlin Heidelberg, 291–299. doi:10.1007/3-540-45061-0_25
- [26] Stephen P. Jordan. [n. d.]. Quantum Algorithm Zoo. <https://quantumalgorithmzoo.org>.
- [27] Andreea-Iulia Lefterovici, Michael Perk, Debora Ramacciotti, Antonio F. Rotundo, S. E. Skelton, and Martin Steinbach. 2025. Beyond asymptotic scaling: Comparing functional quantum linear solvers. arXiv:2503.21420 [quant-ph] <https://arxiv.org/abs/2503.21420>
- [28] Shayan Najd and Simon Peyton Jones. 2017. Trees that Grow. *JUCS - Journal of Universal Computer Science* 23, 1 (2017), 42–62. doi:10.3217/jucs-023-01-0042 arXiv:<https://doi.org/10.3217/jucs-023-01-0042>
- [29] Giacomo Nannicini. 2022. Fast quantum subroutines for the simplex method. arXiv:1910.10649 [quant-ph] <http://arxiv.org/abs/1910.10649>
- [30] Miloš Prokop, Petros Wallden, and David Joseph. 2025. Grover’s Oracle for the Shortest Vector Problem and Its Application in Hybrid Classical–Quantum Solvers. *IEEE Transactions on Quantum Engineering* 6 (2025), 1–15. doi:10.1109/tqe.2024.3501683
- [31] Norman Ramsey and Avi Pfeffer. 2002. Stochastic lambda calculus and monads of probability distributions. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon) (POPL ’02). Association for Computing Machinery, New York, NY, USA, 154–165. doi:10.1145/503272.503288
- [32] Troels F Rønnow, Zhihui Wang, Joshua Job, Sergio Boixo, Sergei V Isakov, David Wecker, John M Martinis, Daniel A Lidar, and Matthias Troyer. 2014. Defining and detecting quantum speedup. *science* 345, 6195 (2014), 420–424.
- [33] André Schrottenloher and Marc Stevens. 2024. Quantum Procedures for Nested Search Problems. *IACR Communications in Cryptology* 1, 3 (2024). doi:10.62056/aee0fhbmo
- [34] P.W. Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. 124–134. doi:10.1109/SFCS.1994.365700
- [35] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (Oct. 1997), 1484–1509. doi:10.1137/s0097539795293172
- [36] Daniel R. Simon. 1997. On the Power of Quantum Computation. *SIAM J. Comput.* 26, 5 (1997), 1474–1483. doi:10.1137/S0097539796298637 arXiv:<https://doi.org/10.1137/S0097539796298637>
- [37] Martin Suchara, John Kubiawicz, Arvin Faruque, Frederic T. Chong, Ching-Yi Lai, and Gerardo Paz. 2013. QuRE: The Quantum Resource Estimator toolbox. In *2013 IEEE 31st International Conference on Computer Design (ICCD)*. 419–426. doi:10.1109/ICCD.2013.6657074
- [38] Wouter Swierstra. 2008. Data types à la carte. *J. Funct. Program.* 18, 4 (July 2008), 423–436. doi:10.1017/S0956796808006758
- [39] M. Szegedy. 2004. Quantum speed-up of Markov chain based algorithms. In *45th Annual IEEE Symposium on Foundations of Computer Science*. 32–41. doi:10.1109/FOCS.2004.53
- [40] Joran Van Apeldoorn, András Gilyén, Sander Gribling, and Ronald de Wolf. 2017. Quantum SDP-solvers: Better upper and lower bounds. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 403–414.
- [41] Sören Wilkening, Andreea-Iulia Lefterovici, Lennart Binkowski, Marlene Funck, Michael Perk, Robert Karimov, Sándor Fekete, and Tobias J. Osborne. 2025. A quantum search method for quadratic and multidimensional knapsack problems. arXiv:2503.22325 [quant-ph] <https://arxiv.org/abs/2503.22325>
- [42] Sören Wilkening, Andreea-Iulia Lefterovici, Lennart Binkowski, Michael Perk, Sándor Fekete, and Tobias J. Osborne. 2024. A quantum algorithm for solving 0-1 Knapsack problems. arXiv:2310.06623 [quant-ph] <https://arxiv.org/abs/2310.06623>
- [43] Christof Zalka. 1999. A Grover-based quantum search of optimal order for an unknown number of marked elements. arXiv:quant-ph/9902049 [quant-ph] <https://arxiv.org/abs/quant-ph/9902049>